

数据结构与算法课程设计

二叉树：104.二叉树的最大深度

1、程序代码

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode()
        : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x)
        : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode* left, TreeNode* right)
        : val(x), left(left), right(right) {}
};

class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == NULL)
            return 0;
        int t = 0;
        queue<TreeNode*> qu;
        qu.push(root);
        while (!qu.empty()) {
            int size = qu.size();
            t++;
            for (int i = 0; i < size; i++) {
                TreeNode* node = qu.front();
                qu.pop();
                if (node->left)
                    qu.push(node->left);
                if (node->right)
                    qu.push(node->right);
            }
        }
        return t;
    }
};

TreeNode* createTree(const vector<int>& arr, int index = 0) {
    if (index >= arr.size() || arr[index] == -1) {
        return nullptr;
    }
    TreeNode* root = new TreeNode(arr[index]);
    root->left = createTree(arr, 2 * index + 1);
    root->right = createTree(arr, 2 * index + 2);
    return root;
}

void deleteTree(TreeNode* root) {
    if (root == nullptr)
        return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}

int main() {
    int t;
    cin >> t;
    while (t--) {
```

```

    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i) {
        cin >> arr[i];
    }
    TreeNode* root = createTree(arr);
    Solution sol;
    int depth = sol.maxDepth(root);
    cout << "二叉树的最大深度是: " << depth << endl;
    // 释放内存
    deleteTree(root);
}

return 0;
}

```

2、运行结果

```

2
7
3 9 20 -1 -1 15 7
二叉树的最大深度是: 3
3
1 -1 2
二叉树的最大深度是: 2

Process returned 0 (0x0)    execution time : 32.374 s
Press any key to continue.

```

3、实习总结与体会

不仅学习到了计算深度，还学习了如何创建二叉树以及释放内存。

(1) 计算深度：使用广度优先搜索（BFS）算法来计算二叉树的最大深度。通过一个队列逐层遍历二叉树，每处理一层，深度加1。

(2) 创建二叉树：根据输入数组递归构建二叉树。数组中的 `-1` 表示空节点。

(3) 释放内存：递归释放二叉树占用的内存，避免内存泄漏。